



Ökolopoly: Case Study on Large Action Spaces in Reinforcement Learning

Raphael C. Engelhardt¹✉, Ralitsa Raycheva¹, Moritz Lange²,
Laurenz Wiskott², and Wolfgang Konen¹

¹ Faculty of Computer Science and Engineering Science,
Cologne Institute of Computer Science, TH Köln, Gummertsbach, Germany
{Raphael.Engelhardt,Wolfgang.Konen}@th-koeln.de

² Faculty of Computer Science, Institute for Neural Computation,
Ruhr-University Bochum, Bochum, Germany
{Moritz.Lange,Laurenz.Wiskott}@ini.rub.de

Abstract. Ökolopoly is a serious game developed by biochemist Frederic Vester with the goal to enhance understanding of interactions in complex systems. Due to its vast observation and action spaces, it presents a challenge for Deep Reinforcement Learning (DRL). In this paper, we make the board game available as a reinforcement learning environment and compare different methods of making the large spaces manageable. Our aim is to determine the conditions under which DRL agents are able to learn this game from self-play. To this goal we implement various wrappers to reduce the observation and action spaces, and to change the reward structure. We train PPO, SAC, and TD3 agents on combinations of these wrappers and compare their performance. We analyze the contribution of different representations of observation and action spaces to successful learning and the possibility of steering the DRL agents' gameplay by shaping reward functions.

Keywords: Deep reinforcement learning · Large action space · Cybernetics · Serious games

1 Introduction

Despite the overwhelming success of Deep Reinforcement Learning (DRL) in the last decade, large action spaces can still pose a challenge for DRL algorithms [7]. The serious game *Ökolopoly* [19] is an example of an environment exhibiting such a large action space. The game has its roots in cybernetics as it aims at teaching the players how to steer circular causal processes. The game in its internationalized computer simulation version *Ecopolicy* is cited as an example for training systemic and long-term thinking in complex, interconnected systems as opposed

This research was supported by the research training group “Dataninja” (Trustworthy AI for Seamless Problem Solving: Next Generation Intelligence Joins Robust Data Analysis) funded by the German federal state of North Rhine-Westphalia.

to linear thinking in terms of immediate, simple cause-effect relationships. International competitions in schools were held (“*Ecopoliciade*”) to train pupils the art of thinking holistically. [1, Sect. 2.2] [12, Sect. 3.2]

The combinatorial explosion of choices quickly leads to a large number of possible game states and a very large action space, making this game an interesting test case for DRL algorithms [7].

In this paper we describe how the board game can be formalized as a Reinforcement Learning (RL) environment following OpenAI Gym [2] standards. Given this implementation, we investigate the following research questions:

- RQ 1 Is it possible for RL agents to learn to play the game Ökolopoly from self-play?
- RQ 2 Which components are essential for successful learning (if any)?
- RQ 3 Can the agent learn to propose valid actions or is it necessary that the environment transforms invalid actions into valid ones?

We explain and experimentally test different methods of approaching such large action spaces. We will show which of these methods are essential for a DRL agent to successfully learn to play the game and to what extent the agent can develop an “understanding” of the underlying game mechanics. Our hope is that the results from this Ökolopoly case study are also useful for other RL problems with large action spaces.

The remainder of the paper is structured as follows: Sect. 2 will discuss related work. In Sect. 3 we briefly describe the game of Ökolopoly. Section 4 contains technical information about how the game is translated from a board game to the domain of RL as well as methods to treat the large observation and action spaces, and different reward functions. Section 6 presents the experimental outcomes. In Sect. 7 we answer the research questions and give a short conclusion.

2 Related Work

Large action spaces have been identified as one of the main challenges in RL [7]. Proposed solution techniques may factorize the action space into binary or ternary subspaces [13], embed the discrete action space in a continuous one [6], or use the technique of action elimination [20]. In our work we will use the first two techniques as well. Instead of action elimination we use action normalization (projection onto valid actions, see Sect. 5.2). While the above-mentioned papers investigate action spaces of size 10^2 – 10^4 , the application studied in this work has an action space of size 10^6 – 10^8 (see Sect. 3.3, depending on whether we use action normalization or not). Huang and Ontañón [11] thoroughly describe and evaluate invalid action masking, a technique to restrict large, discrete action spaces to valid actions only. This is achieved by considering the logits produced by policy gradient algorithms and substituting those corresponding to invalid actions by a large negative number.

Serious games in biology [17] and ecology [12] have a long tradition and are often used for educational purposes or for collective problem-solving in science,

as in *Foldit* [3]. The use of RL for serious games is an emerging research topic [5]. Dobrovsky et al. [4] use interactive DRL to balance the transfer of knowledge and the entertainment in serious games based on the context information from gameplay. Another example are rehabilitation serious games [10] where an RL-based approach is used to modify the difficulty of the rehabilitation exercises.

The game of Ökolopoly has—to the best of our knowledge—not been learned successfully by RL methods before.

3 The Game of Ökolopoly

Designed by Frederic Vester and made available as board game in 1984 [18, 19], the game of Ökolopoly aims to raise awareness for and deepen the understanding of acting in systems of complex interdependencies.

The game is conceived as a single-player, turn-based strategy game. It models the state of the imaginary country of *Kybernetien* with scores on eight interacting departments or fields (such as Population, Quality of Life or Environment). The player’s task is to lead the country to success by cleverly distributing available action points to the fields and developing an understanding of the underlying interdependencies. The fields are described in Table 1 with their minimal, maximal, and starting values.

Table 1. The eight fields, number of rounds played, and available action points determining the state of the game. Five of these fields are directly *actionable*, i.e., they may receive action points.

Field	Min	Max	Start Value	Actionable
Sanitation	1	29	1	yes
Production	1	29	12	yes
Environment	1	29	13	
Education (e)	1	29	4	yes
Quality of Life (q)	1	29	10	yes
Population Growth (g)	1	29	20	yes
Population (b)	1	48	21	
Politics (p)	−10	37	0	
Rounds (r)	1	30	1	
Action Points	1	36	8	

3.1 One Turn of the Game

In each turn (or timestep in RL terms) the player chooses how to distribute the available action points among the five fields Sanitation, Production, Education, Quality of Life, and Population Growth, so that the respective field values are

incremented by those action points. Only for the field Production the player may also choose to diminish the value, at the cost of action points, too. There is no minimum value of action points to use, i.e., the player may choose to save action points for the next round.

Once the action points are distributed, certain interdependency functions between the fields, e.g., $G_j(x)$, $j = 1, \dots, 4$ for the field population growth g , where x is any of the other fields, give rise to a number of automatic adjustments (feedback effects). For example, if Education is $e = 19$, Population Growth changes by $G_1(e) = +3$. The interdependency functions are deterministic, but complex and hard to memorize for a human player. Finally, the action points available for the next round are assigned following the interdependency functions, the round counter is increased by one, and the round ends.¹

For higher values of Education e , the interdependency function $G_1(e)$ may exhibit a number preceded by $\pm$: in those cases the player can choose to diminish or increase the field Population Growth g in the given range at no additional cost of action points (e.g., if $G_1(e) = \pm 3$ then any choice of $\Delta g \in \{-3, -2, -1, 0, 1, 2, 3\}$ is allowed). The reasoning behind this rule is that a sufficiently educated population is able to steer its growth.

3.2 End of the Game

When one or more fields leave the allowed range (either due to the distribution of action points or due to the automatic adjustments thereafter), or when 30 rounds are played, the game ends. At the end of the game, the balance score B is computed as a function of the field Politics p , the value of the interdependency function $D(q)$, which is monotonically rising with Quality of Life q , and the number of played rounds r :

$$B(p, q, r) = \begin{cases} \frac{10[p+3D(q)]}{r+3} & \text{if } 10 \leq r \leq 30 \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

This means that a balance score of 0 is given, if the condition $10 \leq r \leq 30$ is not met. The game instructions define a score of over 20 as exceptionally good.

3.3 Observation and Action Spaces

Given Table 1, the observation space allows for $29^6 \cdot 48^2 \cdot 36 \cdot 30 \approx 1.48 \times 10^{15}$ different states.

When distributing $a \in \{1, \dots, 36\}$ available action points to the five fields, there are in principle $(a+1)^5$ possible combinations. Since a player cannot distribute more action points than available, the number of valid combinations is much smaller. Counting the number of valid and the number of possible combinations for all values of a , we find that there are 9.7×10^6 valid combinations, which are only 1.1% of all 9.1×10^8 possible combinations.

¹ An advanced version of the game provides optional “event cards” to be drawn every five rounds. We ignore this advanced version in our implementation.

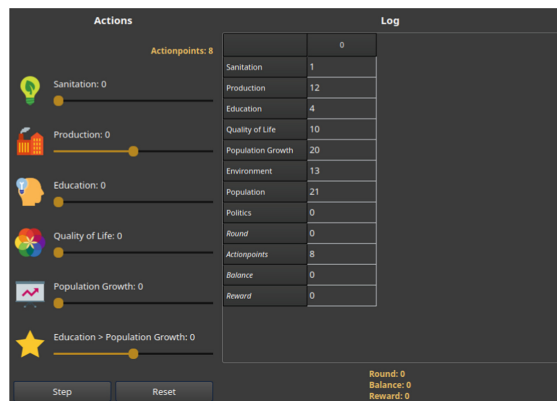


Fig. 1. GUI for the RL environment

This poses two challenges for any DRL agent: Firstly, even when restricting the agent to valid actions (e.g., by sum normalization, see Box action wrapper in Sect. 5.2), there is still a large number of 9.7×10^6 options. Secondly, if we give the agent no information on whether a possible action is valid or not (no action wrapper), it has to learn by reinforcement feedback to suggest valid combinations (otherwise the episode will terminate immediately). This is a demanding task given the small percentage of only 1.1% valid combinations.

4 Implementation of the Game

In this section we briefly describe how the board game was implemented as an OpenAI Gym [2] compatible RL environment. The code and GUI for human-play (Fig. 1) were adapted from [15]. Implementation, experiments, and additional material are available on Github².

4.1 Representation of the Observation Space

The observation space is internally represented as a ten-dimensional object of class `MultiDiscrete` containing the values of the eight fields, the number of rounds played, and the currently available action points. The agent has therefore full access to all information visible to the human player of the board game. As different ranges are allowed in the different dimensions of the observation space (the field `Politics` can even contain negative values which are not supported by `MultiDiscrete`), allowed states are shifted accordingly (see first row of Table 2).

4.2 Representation of the Action Space

In a similar way the action space is encoded as a six-dimensional `MultiDiscrete` object containing the number of action points assigned to each of the five fields.

² https://github.com/WolfgangKonen/oekolopoly_v1.

The sixth number accounts for the possibility of modifying Population Growth by up to ± 5 points according to the value of $G_1(e)$ (see end of Sect. 3.1).

4.3 Reward Functions

The basic reward function merely implements Eq. 1. This requires the agent to perform a long streak of profitable actions, which is difficult to find by exploration, before receiving any learning signal (only after the terminal step and if it occurs after at least ten rounds). For this reason, we implemented and tested different auxiliary reward structures, which additionally assign a reward after each intermediate step. These dense reward functions are described in detail in Sect. 5.3.

5 Methods

To assess the impact of different ways to handle the large observation and action spaces and different reward structures on the success of training, we performed experiments with different combinations of DRL algorithms and wrappers we describe in the following. A summary of the different spaces and their implementation is given in Table 2.

5.1 Observation Wrappers

We consider three different observation wrapper choices that should enable the algorithms to digest the huge observation space.

None. We treat the observation space as a `MultiDiscrete` object.

Box Observation Wrapper. In the case of `MultiDiscrete` observations, the DRL agent does not have an intrinsic concept of distance between possible values in each dimension of the observation space. To mitigate this problem, the Box observation wrapper represents each of the eight fields, the played rounds, and the available action points as a value in a continuous `Box` reaching from the minimum to the maximum of the respective observation.

Simple Observation Wrapper. This observation wrapper subdivides each dimension into just three possible values: *low*, *medium*, and *high*. This way each field, the current number of rounds played, and the available action points are represented each by one value in $\{0, 1, 2\}$. The state space is thereby reduced to $3^{10} = 59\,049$ different states. The observation wrapper is implemented as a ten-dimensional `MultiDiscrete` object.

5.2 Action Wrappers

Similarly, we implemented and tested three different choices for the action wrapper to simplify the action space.

None. The action space is the unaltered `MultiDiscrete` object from Sect. 4.2. Since there is no mechanism translating actions into legal moves, the validity of an action is not ensured. In fact, the overwhelming majority of points in this action space do not correspond to valid moves.

Box Action Wrapper. This wrapper transforms the discrete action space into a continuous one of type `Box`. The elements of this six-dimensional vector range from $[0, -1, 0, 0, 0, -1]$ to $[1, 1, 1, 1, 1, 1]$ and have the meaning described in Sect. 4.2. If more than the available action points should be distributed according to the tentative action vector $\mathbf{a}'$, the values are normalized by their absolute sum, multiplied with the currently available action points n , and rounded to the next integer:

$$a_i = \left\lfloor \frac{a'_i}{\sum_{i=0}^4 |a'_i|} \cdot n + 0.5 \right\rfloor \quad \text{for } i = 0, \dots, 4$$

If, due to rounding effects, the sum of action points a_i still exceeds n , the highest element of vector $\mathbf{a}$ is decreased by one³.

Simple Action Wrapper. Analogous to the Simple observation wrapper, this action wrapper reduces the number of available discrete actions by dividing the number of available action points into three equal or near-equal parts. These blocks of action points can then be distributed among the five different fields in the game. This distribution is encoded as a six-digit string: The first five digits from the left assume values in $\{0, 1, 2, 3\}$ which represent the number of action point partitions assigned to the respective field (the sum of the first five digits may therefore not exceed 3); the rightmost digit encodes whether action points are added (0) or deducted (1) from Production⁴. In total there are 77 such strings to represent legal moves. The action space is implemented as two-dimensional `MultiDiscrete` object whose first element contains the index of one of the 77 legal moves while the second one contains the possibility of modifying Population Growth by up to ± 5 . The total number of possible actions is therefore reduced to $77 \cdot 11 = 847$. On the other hand, actions are less precise. In certain situations, this can lead to premature episode termination, which could have been avoided by a finer-grained distribution. The Simple action wrapper mimicks a possible human strategy of being less precise but actionable in unknown complex environments.

5.3 Reward Functions

We trained DRL agents using the score defined by the rules of the board game (see Sect. 3.2) as well as two other reward functions which alleviate mentioned problems related to this sparse reward structure.

³ The sixth dimension $a'_5 \in [-1, 1]$ (modifier for Population Growth g , Sect. 3.1) is multiplied by 5, rounded, and then appended to $\mathbf{a}$.

⁴ As an example the string 020101 encodes the following distribution of action points: one third of the action points are added to Education, two thirds of the action points are deducted (rightmost digit is 1) from Production.

Balance. At intermediate timesteps no reward is given. Only the final step yields a reward B computed according to Eq. 1.

PerRound. This reward wrapper issues an additional constant reward R_c after each intermediate step. After the episode’s last step the known reward B from Eq. 1 is added to the return. This should be an incentive for the agent to reach a higher number of rounds, as the rules of the game suggest that between 10 and 30 rounds should be played. Different values for R_c will be investigated. If not stated otherwise, we use $R_c = 0.5$.

Heuristic. A heuristic that keeps Production and Population at healthy, intermediate values (15 and 24 respectively) empirically seemed to be a good strategy. To this end, the following auxiliary reward R is assigned after each intermediate step:

$$R = s \cdot (R_{\text{prod}} + R_{\text{pop}})$$

$$\text{with } R_{\text{prod}} = 14 - |15 - V_{\text{prod}}|$$

$$R_{\text{pop}} = 23 - |24 - V_{\text{pop}}|$$

After the last step of each episode, the usual balance B from Eq. 1 is given as reward. The scaling factor s allows to steer the agent to maximize the auxiliary reward R and therefore the number of rounds (for higher values such as $s = 1$) or to maximize the balance B (for smaller values of s , see Fig. 7). Where not explicitly stated otherwise, we use $s = 1$.

Table 2. Overview of action and observation space wrappers and their representation

Space	Wrapper	Object
Observation	None	<code>MultiDiscrete([29,29,29,29,29,29,48,48,31,37])</code>
	Box	<code>Box(low=[1,1,1,1,1,1,-10,0,0], high=[29,29,29,29,29,29,48,37,30,36])</code>
	Simple	<code>MultiDiscrete([3,3,3,3,3,3,3,3,3,3])</code>
Action	None	<code>MultiDiscrete([29,57,29,29,29,11])</code>
	Box	<code>Box(low=[0,-1,0,0,0,-1], high=[1,1,1,1,1,1])</code>
	Simple	<code>MultiDiscrete([77,11])</code>

5.4 DRL Algorithms

For the experiments we trained agents with the on-policy algorithm PPO [16] and the off-policy algorithms TD3 [8] and SAC [9] using their implementation provided by the Python DRL framework Stable-Baselines3 [14]. We use the default hyperparameters of SB3; for the TD3 agents, Gaussian action noise with $\sigma = 0.1$ is applied. While all mentioned algorithms are suited to handle `MultiDiscrete` and `Box` observation spaces, not all are compatible with `MultiDiscrete` action spaces (see Table 3). The computational effort varies noticeably depending on the algorithm. The elapsed real time to train an agent (no observation wrapper,

Box action wrapper, PerRound reward with $R_c = 0.5$) for 800 000 timesteps was 680.0 ± 2.1 s for PPO, $13\,216.2 \pm 115.8$ s for SAC, and $11\,786.5 \pm 55.5$ s for TD3 (three repetitions)⁵.

Table 3. Compatibility of mentioned algorithms and used action and observation space representations.

Algorithm	Observation Space		Action Space	
	MultiDiscrete	Box	MultiDiscrete	Box
PPO	✓	✓	✓	✓
SAC	✓	✓	✗	✓
TD3	✓	✓	✗	✓

5.5 Experimental Setup

As a consequence of Table 2 and Table 3, not every combination of wrapper and algorithm is possible: we have $3 \cdot 3 \cdot 3 = 27$ combinations for PPO, but only 9 combinations for SAC and TD3 (only Box action wrappers). For our experiments we train DRL agents with all available combinations of observation and action wrappers and reward functions for 800 000 timesteps while logging the balance B (Eq. 1), the number of played rounds r , and the reward as seen by the DRL agent. After each completed training episode the agent plays one evaluation episode using deterministic predictions according to its current training state. As a measure of the agents’ performance we compute the averages and standard deviations of balance B and played rounds r of the last 1 000 evaluation episodes.

We investigate the stability of the training process with respect to initialization of the agents’ networks, the role of the different methods in handling observation and action space, the impact of reward functions, and the strategy of single agents.

6 Results

After inspecting the training history we examine our results regarding the role of different representations of observation and action space as well as different reward functions. We touch upon the possibility of steering the agents’ focus by suitable choices of rewards and provide more insights regarding the agents’ performance in terms of balance and played rounds.

6.1 Stability of Training

To assess how critical the random initialization of the DRL agent’s networks is, as an example we trained 5 PPO agents on the combination Box observation

⁵ Timing experiments were performed on a system with Intel® Core™i7-1185G7 CPU and 16 GB RAM.

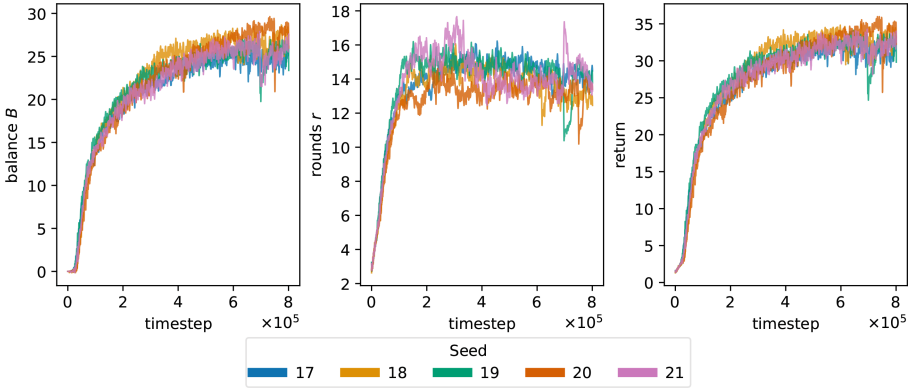


Fig. 2. Training curves of 5 differently seeded PPO agents (Box observation, Box action wrapper, PerRound reward $R_c = 0.5$) encoded by color. (For better visualization a Savitzky-Golay smoothing filter was applied.)

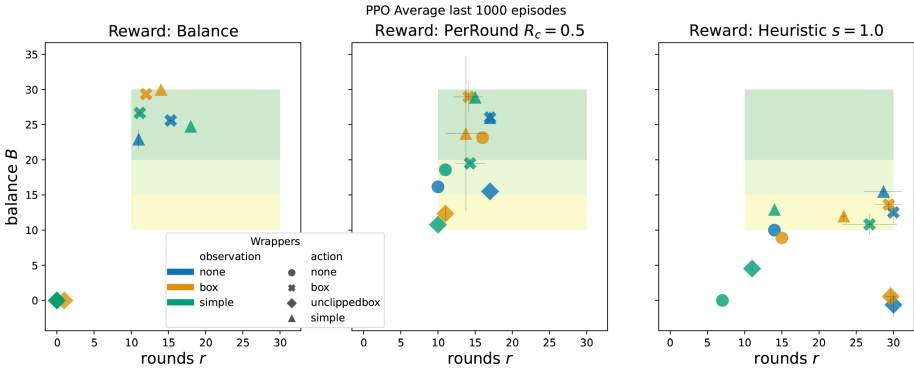


Fig. 3. Average and standard deviation of number of played rounds and balance for the last 1000 evaluation episodes of **PPO** agents. Different reward functions are shown in separate plots, while observation and action wrapper are encoded by color and marker. The areas marked by colored backgrounds represent scores considered as increasingly good by the rules of the game. (N.b. at (0,0) the combination without observation wrapper and UnclippedBox action wrapper (blue diamond), as well as all combinations without action wrapper (circles) are hidden below the green diamond marker). (Color figure online)

wrapper, Box action wrapper, and PerRound reward $R_c = 0.5$ under identical conditions except for a different seed of the DRL algorithm. As can be seen in Fig. 2, the initialization has at most only marginal effects on the training process and especially on the outcome.

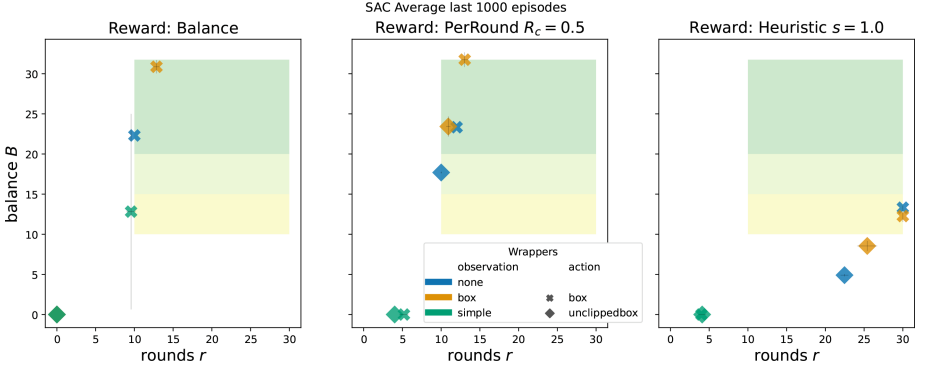


Fig. 4. Average and standard deviation of number of played rounds and balance for the last 1000 evaluation episodes of **SAC** agents. The plots are structured as in Fig. 3. Tables 2 and 3 show why this plot contains fewer results. (N.b. the three diamond markers at (0, 0) are overlapping in the leftmost plot).

6.2 Impact of Wrappers

To compare the training outcomes of different combination of wrappers, we examine the average and standard deviation of the agents’ performance across the last 1000 evaluation episodes during training in terms of played rounds and balance. The results are shown in Fig. 3 (PPO) and Fig. 4 (SAC).

Without ensuring valid actions by the use of action wrappers and with only sparse reward, the agents do not receive a sufficient learning signal and consequently fail (three overlaying round markers at (0, 0) in the left plot of Fig. 3). With action wrappers intrinsically allowing only valid moves, the agents accumulate in a specific region of the multiobjective plot with exceptionally high score after having played between 10 and 20 rounds. If PPO agents are rewarded with a fixed positive amount R_c after each intermediate step, they are able to handle even large multidiscrete observation and action spaces, learn legal moves, and find a suitable strategy, as shown in the central plot. While a clear clustering by color or marker could not be observed here, the right plot shows the agents’ difficulties to find suitable actions by the accumulation of round markers in the lower left side.

While PPO agents exhibit an overall remarkably good performance (Fig. 3), SAC agents using the Simple observation wrapper seem to struggle (Fig. 4). We also trained TD3 agents with default hyperparameters and Gaussian action noise with $\sigma = 1$. Since their results were inferior to PPO and SAC (only some TD3 combinations resulted in a balance above 10; most combinations have a balance $B \approx 0$ and rounds $r < 10$), we do not show these results.

The action wrappers serve a dual purpose: not only do they reduce the number of available actions, they also ensure only valid actions can be performed. To investigate which effect is predominant, all PPO and SAC agents were also trained using a modified Box action wrapper. This *UnclippedBox* action wrap-

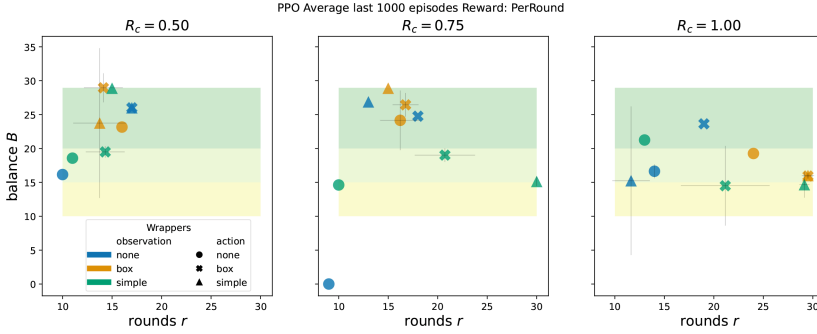


Fig. 5. Effect of different values of constant per-round reward R_c on PPO agents

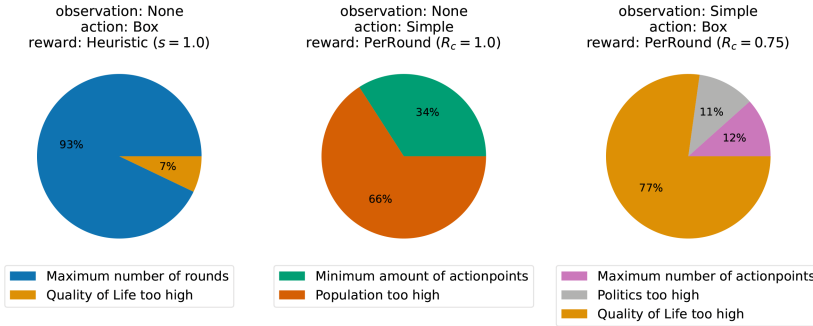


Fig. 6. Reasons for episodes’ termination of the last 1000 evaluation episodes during training. Shown is a small subset of PPO agents from the set of about 140 pie charts.

per still represents the discrete action space as a continuous box space, but it does not ensure that the intended distribution of action points is possible (i.e., does not exceed the number of available action points). Figures 3 and 4 show how the additional task of learning valid distributions of action points negates any success in case of a sparse reward. With dense rewards, *UnclippedBox* leads to somewhat reduced performance but it is noteworthy that the agents learn to perform valid actions.

Investigating the agent’s gameplay further, a variety of different reasons for episode’s termination emerge, many of which are “positive” in the sense that they leave the range of possible values in a direction that would commonly be considered desirable (e.g., “Quality of Life too high”). For three agents taken as examples, Fig. 6 shows statistics of reasons for termination of the last 1000 evaluation episodes during training⁶.

⁶ The full set of pie charts is available in the [Github repository](#).

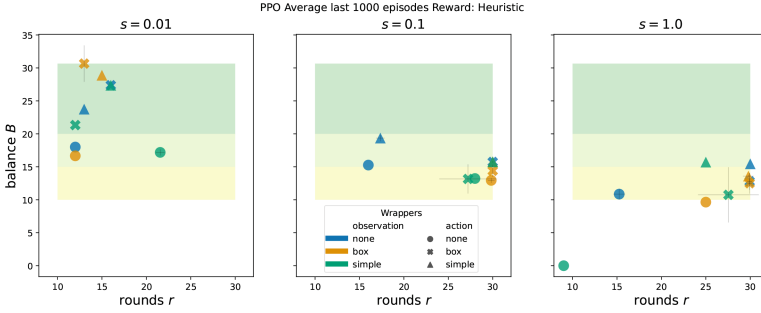


Fig. 7. Effect of different scaling of Heuristic reward on PPO agents.

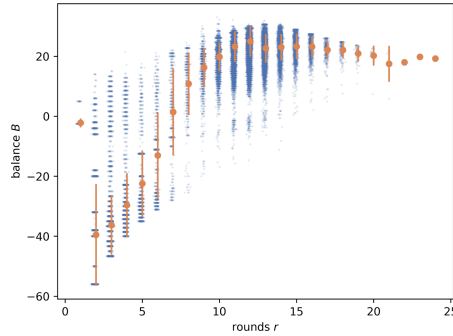


Fig. 8. Correlation of rounds r and balance B . Blue dots mark every pair (r, B) encountered by the PPO agent during training for the combination Box observation wrapper, Box action wrapper, reward Balance. Orange dots mark the average for every number of rounds. (Color figure online)

6.3 Reward Shaping

The importance of a suitable reward function is well-known among RL practitioners. This holds true also in the case of Ökolopoly. Not only does the reward function have arguably the biggest effect on overall performance (see Fig. 3), but also do the parameters of the single reward function itself play an important role. In Fig. 5 we show the results of comparing three different values of the constant per-round reward R_c . By changing this value we can tune the agent towards surviving more rounds r (in case of higher values R_c) on one side or towards prioritizing a higher balance B on the other.

Similarly, Fig. 7 shows the results for different values of the scaling factor s of the Heuristic reward.

6.4 Optimal Balance and Rounds

Looking at Fig. 5, one might ask why in the case $R_c = 0.5$ (where the incentive is more on balance) the high balance results are not possible in combination

with a higher number of rounds. The average episode length does not exceed 17 rounds. A similar observation is made in the ‘Reward: Balance’ plots of Fig. 3 and Fig. 4.

The correlation is visualized in Fig. 8: Firstly, the balance B (for rounds $r < 10$ only hypothetical) shows a local optimum roughly at $r \approx 15$. Secondly, the number of episodes with $r \in [10, 15]$ is about 2000 times higher than the number of episodes with $r \in [20, 25]$, meaning that the average in Fig. 5, $R_c = 0.5$, is likely to be below $r \approx 15$. The agent has learned that it is not detrimental to stop an episode after 10 – 15 rounds, because a longer lasting episode would not improve (or even diminish) B . This is why the agent often stops quite early (after surpassing the required 10 rounds) unless a special incentive (e.g., $R_c = 1$) tells it otherwise.

7 Conclusion

In this article we described how we made the cybernetic board game Ökolopoly accessible as an RL environment. Various ways of simplifying the large observation and action space as well as different reward functions have been investigated. We were able to show that in many of our combinations, the DRL agents could successfully learn how to deal with the rather complex interdependencies of the game. Our results show that DRL agents can consistently learn to play the game from self-play (RQ 1). However, at least one of the following conditions have to be met in order to be successful (RQ 2): (i) an action wrapper that inherently maps to valid actions (Box or Simple) or (ii) a dense reward function (PerRound or Heuristic) are applied.

If neither (i) nor (ii) is fulfilled, no learning occurs in this large action space. We have shown that within the Box action wrapper not the mapping to continuous space but the clipping to valid actions is essential. We got the interesting result that even in the absence of action wrappers, the agent can learn by self-play to propose valid actions, given a dense reward structure (RQ 3).

We invite everyone to use our new RL environment Ökolopoly, which is made available as a benchmark for researchers, to test their methods for handling large observation and action spaces.

References

1. Bosch, O., Nguyen, N., Sun, D.: Addressing the critical need for “new ways of thinking” in managing complex issues in a socially responsible way. *Bus. Syst. Rev.* **2**, 48–70 (2013)
2. Brockman, G., et al.: OpenAI Gym (2016). <https://doi.org/10.48550/arXiv.1606.01540>
3. Cooper, S., et al.: Predicting protein structures with a multiplayer online game. *Nature* **466**(7307), 756–760 (2010). <https://doi.org/10.1038/nature09304>
4. Dobrovsky, A., Borghoff, U.M., Hofmann, M.: Improving adaptive gameplay in serious games through interactive deep reinforcement learning. *Cogn. Infocommun. Theory Appl.* **13**, 411–432 (2019)

5. Dobrovsky, A., Wilczak, C.W., Hahn, P., Hofmann, M., Borghoff, U.M.: Deep reinforcement learning in serious games: analysis and design of deep neural network architectures. In: Moreno-Díaz, R., Pichler, F., Quesada-Arencibia, A. (eds.) EUROCAST 2017. LNCS, vol. 10672, pp. 314–321. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-74727-9_37
6. Dulac-Arnold, G., et al.: Deep reinforcement learning in large discrete action spaces (2015). <https://doi.org/10.48550/arXiv.1512.07679>
7. Dulac-Arnold, G., et al.: Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Mach. Learn.* **110**(9), 2419–2468 (2021). <https://doi.org/10.1007/s10994-021-05961-4>
8. Fujimoto, S., van Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. In: Dy, J., Krause, A. (eds.) Proceedings of the 35th International Conference on Machine Learning, PMLR, vol. 80, pp. 1587–1596 (2018)
9. Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: Dy, J., Krause, A. (eds.) Proceedings of the 35th International Conference on Machine Learning, PMLR, vol. 80, pp. 1861–1870 (2018)
10. Hornak, D., Jascur, M., Ferencik, N., Bundzel, M.: Proof of concept: using reinforcement learning agent as an adversary in serious games. In: 2019 IEEE International Work Conference on Bioinspired Intelligence, pp. 111–116 (2019)
11. Huang, S., Ontañón, S.: A closer look at invalid action masking in policy gradient algorithms. In: The International FLAIRS Conference Proceedings, vol. 35 (2022). <https://doi.org/10.32473/flairs.v35i.130584>
12. Nguyen, N.C., Bosch, O.J.H.: The art of interconnected thinking: starting with the young. *Challenges* **5**(2), 239–259 (2014). <https://doi.org/10.3390/challe5020239>
13. Pazis, J., Parr, R.: Generalized value functions for large action sets. In: Proceedings of the 28th International Conference on International Conference on Machine Learning, pp. 1185–1192 (2011)
14. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N.: Stable-baselines3: reliable reinforcement learning implementations. *J. Mach. Learn. Res.* **22**(268), 1–8 (2021)
15. Raycheva, R.: Erstellung eines custom environments in OpenAI Gym für das Spiel Ökolopoly. Technical report, TH Köln (2021)
16. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms (2017). <https://doi.org/10.48550/arXiv.1707.06347>
17. Teixeira, J.d.S., Angeluci, A.C.B., Junior, P.P., Martin, J.G.P.: ‘Let’s play?’ A systematic review of board games in biology. *J. Biol. Educ.* 1–20 (2022). <https://doi.org/10.1080/00219266.2022.2041461>
18. Vester, F.: Der blaue Planet in der Krise. *Gewerkschaftliche Monatshefte* **39**(12), 713–773 (1988)
19. Vester, F.: Ökolopoly: das kybernetische Umweltspiel. Studiengruppe für Biologie und Umwelt (1989)
20. Zahavy, T., Haroush, M., Merlis, N., Mankowitz, D.J., Mannor, S.: Learn what not to learn: action elimination with deep reinforcement learning. In: Bengio, S., et al. (eds.) *Advances in Neural Information Processing Systems*, vol. 31 (2018)